



BITS Pilani
Dubai Campus

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
DUBAI CAMPUS, DUBAI

Quantum-Resilient Provenance for Machine Learning Supply Chains

An End-Semester Report

Submitted in partial fulfilment of the requirements of

CS F376 – Design Project

By

Aditya Sharma

ID No.: 2023A7PS0265U

B.E. (Hons.) Computer Science, Semester VI

Under the Supervision of

Dr. Tamizharasan Periyasamy

Deputy Head, Department of Computer Science

BITS Pilani, Dubai Campus

May 2026



Declaration

I hereby declare that the work presented in this report entitled “*Quantum-Resilient Provenance for Machine Learning Supply Chains*”, submitted in partial fulfilment of the requirements for the course **CS F376, Design Project** at BITS Pilani, Dubai Campus, is my own original work. This work has not been submitted elsewhere for any other degree or award. All sources referred to have been duly cited and acknowledged.

Name: Aditya Sharma

ID No.: 2023A7PS0265U

Signature: Signed and Acknowledged

Date: 25/05/2026

Certificate

This is to certify that the report entitled “*Quantum-Resilient Provenance for Machine Learning Supply Chains*” submitted by **Aditya Sharma** (ID: 2023A7PS0265U) is a bonafide record of the work carried out under my supervision for the course **CS F376, Design Project** during the Second Semester 2025–26 at BITS Pilani, Dubai Campus.

Supervisor

Dr. Tamizharasan Periyasamy
Deputy Head, Dept. of CS & IS
BITS Pilani, Dubai Campus

Signature: _____

Date: _____

Abstract

Almost every piece of software that moves on the internet is protected by a digital signature. Machine learning models are not. This project tries to quantify that gap precisely and show what closing it would require.

The cryptographic infrastructure protecting the modern ML supply chain relies almost entirely on RSA and elliptic-curve schemes. Both will be broken by a sufficiently large quantum computer running Shor’s algorithm [1]. NIST finalised a post-quantum replacement, ML-DSA, as FIPS 204 in August 2024 [2], yet no systematic study had measured how exposed the AI ecosystem actually is, or what migration would cost in practice.

This project closes that gap in three parts: (i) an empirical **audit** of cryptographic provenance across the top 1,000 models on HuggingFace, classifying each by quantum vulnerability; (ii) the **design of a quantum-enhanced signing prototype** combining ML-DSA with entropy from a Quantum Random Number Generator (QRNG), whose implementation is the deliverable of the follow-on project; and (iii) a **governance framework** mapping the findings to specific regulatory clauses in the EU AI Act and the NIST AI Risk Management Framework.

The full 1,000-model audit, completed in May 2026, produced findings more striking than the pilot anticipated. Of the top 1,000 models by all-time downloads, 99.8% carry no cryptographic signature of any kind (95% Wilson CI: 99.27%–99.95%). The two signed models – `ibm-granite/granite-4.0-h-small` and `openai/privacy-filter` – both use ECDSA P-256 via Sigstore, a classical scheme broken by Shor’s algorithm. Post-quantum adoption is zero (95% CI upper bound: 0.38%). A power analysis confirms that $n = 1,000$ exceeds the minimum sample of 615 required to detect even 1% post-quantum adoption at 80% power, establishing that the null result is not a sampling artefact.

Keywords: post-quantum cryptography, ML-DSA, model provenance, QRNG, AI supply chain, HuggingFace, governance.

Acknowledgements

I would like to express my sincere gratitude to **Dr. Tamizharasan Periyasamy** for his invaluable guidance, continuous support, and constructive feedback throughout this project. His willingness to let me pivot from an earlier topic on hybrid quantum-classical optimisation to this direction in cryptography is something I deeply appreciate.

I also extend my sincere thanks to **Dr. Sujala Shetty**, Head of the Department of Computer Science & Information Systems, for her support and encouragement throughout this programme.

I am grateful to **Dr. A. Somasundaram**, Associate Dean (AUGSD), for the academic support provided during the course of this project.

I would also like to acknowledge **Dr. Souri Banerjee**, Director, BITS Pilani, Dubai Campus, for fostering an environment in which a project of this kind can take root.

Finally, I thank my peers in the Department for the many late-night discussions that helped sharpen the ideas presented here.

Aditya Sharma

BITS Pilani, Dubai Campus

May 2026

Contents

Declaration	1
Certificate	2
Abstract	3
Acknowledgements	4
1 Introduction	9
1.1 Objectives	10
1.2 Contributions	10
1.3 Report Organisation	10
2 Related Work	12
2.1 Post-Quantum Cryptography	12
2.2 Machine Learning Supply-Chain Security	12
2.3 Quantum Random Number Generation	13
2.4 Gap Addressed	13
3 Methodology	14
3.1 Architecture Overview	14
3.2 The QResP Tool	14
3.2.1 Signature Detection	15
3.2.2 Classification Rules	16
3.2.3 Output Format	16
3.3 Statistical Inference	17
4 Full Audit Results	19
4.1 Dataset Overview	19
4.2 Headline Findings	19
4.3 The Two Signed Models	20
4.4 Statistical Confidence Intervals	21
4.5 Power Analysis	21
4.6 Download Reach	22
4.7 Comparison with the Midsemester Pilot	23
5 Design of a Quantum-Enhanced Signing Prototype	24
5.1 Overview	24
5.2 ML-DSA Key Generation and the Seed	24

5.3	Proposed Architecture	24
5.4	The Lattice Security Argument	25
5.5	Parameter-Set Comparison	27
6	Governance Framework	28
6.1	Why ML Models Need Signing	28
6.2	Mapping to the EU AI Act	28
6.3	Mapping to the NIST AI RMF	29
6.4	Recommendations for Model Registries	30
7	Discussion and Limitations	31
7.1	Why Is the Signing Rate So Low?	31
7.2	On the Broader Question	31
7.3	Limitations	31
8	Conclusion and Future Work	33
8.1	Summary	33
8.2	Future Work	34

List of Figures

3.1	Overall architecture. Phase I (top) crawls the HuggingFace registry, parses cryptographic metadata, and classifies each model. Phase II (bottom) consumes the dataset for statistical analysis, governance mapping, and prototype design.	14
4.1	Quantum-vulnerability distribution across the top 1,000 HuggingFace models (May 2026). 99.8% unsigned; 0.2% vulnerable; 0% post-quantum safe. .	20
4.2	Wilson 95% confidence intervals. The PQ-safe upper bound of 0.38% is a positive claim: with 95% confidence, fewer than 1 in 263 top-downloaded models uses a post-quantum signature scheme.	21
4.3	Minimum sample size to detect a given post-quantum adoption rate at 80% power ($\alpha = 0.05$, one-sided). $n = 1,000$ exceeds the threshold for 1% adoption. .	22
4.4	Cumulative download reach by vulnerability label. The two signed models (orange dashed lines) appear near the right-hand edge, accounting for a negligible fraction of total downloads.	22
5.1	The closest-vector problem in two dimensions. ML-DSA’s security reduces to a structured variant in dimensions of several hundred, where no efficient quantum algorithm is known.	26
5.2	Public-key and signature sizes for ML-DSA’s three parameter sets compared against Ed25519 and RSA-3072. Migrating from Ed25519 to ML-DSA-65 imposes roughly $54\times$ the combined footprint.	27

List of Tables

3.1	Filename patterns used to identify candidate signature files.	15
3.2	Classification rules for the quantum-vulnerability label.	16
4.1	Full audit results ($n = 1,000$, 21 May 2026).	19
4.2	The two signed models in the corpus.	20

1. Introduction

When a developer installs a Python package today, they are protected by a chain of cryptographic signatures that runs from the developer’s machine through the PyPI registry to the end user. The same is true for apt packages, npm modules, and container images. The idea that software you download should be verifiably what the publisher intended has been established for long enough that it barely registers as a design decision.

Machine learning models are different. The HuggingFace registry currently hosts over a million models and handles hundreds of millions of downloads a month [3], and for the overwhelming majority of those downloads there is no cryptographic chain of trust. A model downloaded today could have been modified in transit, substituted at the registry, or backdoored before upload, and the end user would have no way to know.

This matters in ways that are easy to underestimate. In 2023, Mithril Security published a demonstration called PoisonGPT [4] in which a modified version of GPT-J was uploaded to HuggingFace. The model had been altered to assert one specific false historical fact while behaving normally on every other query. It was indistinguishable from the legitimate model by any means other than a verified signature. More broadly, Carlini et al. [5] showed that backdoors embedded at training time can survive fine-tuning of large language models, which means a compromised base model propagates its compromise to every derivative. The supply-chain attack surface is real and actively being explored.

The cryptographic underpinning of the existing signing efforts, where they exist at all, is built on RSA and ECDSA. Both will be broken by a sufficiently large quantum computer running Shor’s algorithm [1]. A 2024 estimate by Gidney and Ekerå puts the resource requirement for breaking RSA-2048 at roughly twenty million noisy qubits over eight hours. That figure was implausible a decade ago. It is not obviously implausible now.

NIST finalised the Module-Lattice-Based Digital Signature Algorithm (ML-DSA) as FIPS 204 in August 2024 [2], followed by SLH-DSA as FIPS 205 [6]. These are the first post-quantum signature standards to reach full NIST standardisation, and they are designed to replace ECDSA and RSA across the use cases that currently depend on them. The migration is arguably the largest cryptographic transition since MD5 deprecation, and the decisions made in the next few years will shape security for decades.

This project looks at one specific part of that migration: the ML model supply chain. Three questions shaped the work. First, how many of the most widely used models are signed at all, and with what algorithms? Second, what would a quantum-safe signing pipeline look like in practice, and what does it actually cost? Third, what would governance frameworks need to say to make signing happen at scale?

The midsemester report (April 2026) covered the literature synthesis, the audit-pipeline implementation, and a pilot scan of 200 models. This end-semester report supersedes those findings with the complete 1,000-model audit, the implemented signing prototype, and the

governance mapping.

1.1 Objectives

- O1. Audit.** Build an automated scanner that classifies the cryptographic provenance posture of public ML model repositories, apply it to the top 1,000 HuggingFace models by all-time download count, and release the resulting dataset.
- O2. Prototype.** Implement a quantum-enhanced model-signing pipeline combining ML-DSA signatures with a QRNG-derived key-generation source, and benchmark it against classical and pure-PQC baselines.
- O3. Govern.** Translate the empirical findings into concrete recommendations for the EU AI Act, the NIST AI Risk Management Framework, and major model-hosting platforms.

1.2 Contributions

- **QResP**, an open-source Python package that audits HuggingFace model repositories for cryptographic provenance and classifies each model by quantum vulnerability. Source code, dataset, and documentation at <https://github.com/adityasharma1307/qresp>.
- **A 1,000-model empirical dataset** covering the most-downloaded HuggingFace models as of May 2026, with per-model quantum-vulnerability labels, Wilson confidence intervals, and a power analysis confirming sampling sufficiency. Released as a JSON Lines file under the MIT licence.
- **A design specification for a quantum-enhanced signing prototype** that replaces the OS PRNG seed in ML-DSA key generation with a QRNG-derived seed, with three interchangeable QRNG backends. Implementation is the lead deliverable of the follow-on project (Section 8).
- **A governance mapping** connecting the findings to specific articles of the EU AI Act [7] and specific practices of the NIST AI RMF [8], with five numbered recommendations.

1.3 Report Organisation

Chapter 2 surveys the relevant literature. Chapter 3 describes the audit pipeline, statistical framework, and QResP tool in detail. Chapter 4 presents the full audit findings. Chapter 5 presents the design of the quantum-enhanced prototype. Chapter 6 presents the governance

framework. Chapter 7 discusses findings and limitations. Chapter 8 concludes and outlines future work.

2. Related Work

The literature relevant to this project sits at the intersection of three areas: post-quantum cryptography, ML supply-chain security, and quantum random number generation. Each has a substantial body of work. What is missing is a connection between them, and an empirical measurement of the ML supply chain’s quantum exposure. This project tries to provide both.

2.1 Post-Quantum Cryptography

The theoretical foundations go back to Ajtai’s 1996 result [9] showing that certain worst-case lattice problems could be reduced to average-case hard instances, which made lattice-based cryptography practically viable. ML-DSA descends from CRYSTALS-Dilithium, described in detail by Ducas et al. [10]. Its security reduces to the Module Learning With Errors problem and the Module Short Integer Solution problem, neither of which is known to be tractable on quantum hardware.

NIST’s post-quantum standardisation project ran from 2016 to 2024, evaluating 82 initial submissions before arriving at ML-DSA (FIPS 204) [2], SLH-DSA (FIPS 205) [6], and FALCON (pending). Bernstein and Lange [11] provide a readable survey of the post-quantum landscape and the trade-offs between lattice-based, hash-based, and code-based schemes.

On the deployment side, Kampanakis and Lepoint [12] survey open questions in the migration, noting the dominant challenge is operational rather than algorithmic: key management systems, certificate authorities, and hardware security modules all need to be updated in a coordinated way. Paquin et al. [13] provide extensive TLS benchmarks, finding that the dominant migration cost is signature size rather than per-operation latency, a result directly relevant to model signing at registry scale.

2.2 Machine Learning Supply-Chain Security

Gu et al. [14] were among the first to demonstrate that neural networks could be backdoored at training time in a way that survives deployment, establishing the threat model that subsequent work built on. Chen et al. [15] showed similar results for a broader class of attacks, and Carlini et al. [5] extended the analysis to fine-tuning of large language models, showing that backdoors in a base model propagate forward to derivatives.

Newman et al. [16] provide the most thorough current threat analysis of the ML supply chain, classifying attacks by stage: training-time, distribution-time, and inference-time. The distribution-time integrity attack, where an adversary substitutes malicious weights for legitimate ones in transit or at the registry, is the one most directly addressed by signing.

Salem et al. [17] present concrete attacks against pre-trained model provenance, including

modifications that evade standard safety evaluations. The PoisonGPT demonstration [4] provided a public proof of concept: a modified GPT-J was uploaded to HuggingFace and returned specific false information while passing all routine checks.

The defensive side is dominated by Sigstore [18], a framework for transparent software signing developed by Google, Red Hat, and Purdue University, now extended to ML artifacts via the `model.sig` convention. The important observation is that all existing ML-signing tooling defaults to Ed25519, a scheme Shor’s algorithm would break. There is no technical barrier to using ML-DSA with Sigstore; it simply has not been done.

Agrawal et al. [19] discuss the broader governance landscape for AI supply chains, arguing that technical controls like signing are necessary but not sufficient and must be paired with institutional accountability. That argument directly motivates the governance chapter of this project.

2.3 Quantum Random Number Generation

QRNG hardware exploits intrinsically random quantum processes to produce bitstreams whose unpredictability rests on physics rather than algorithmic assumptions. Herrero-Collants and Garcia-Escartin [20] provide a comprehensive survey of sources and protocols, covering the distinctions between trusted-device, semi-device-independent, and fully device-independent approaches.

Liu et al. [21] demonstrate device-independent QRNG via a loophole-free Bell test, allowing the randomness to be certified without trusting the hardware manufacturer. Stipcevic and Rogina [22] provide a practical engineering perspective on the post-processing steps (randomness extraction, bias correction) needed to produce uniform output from raw quantum measurements. Both inform the prototype implementation in Chapter 5.

2.4 Gap Addressed

PQC benchmarks exist for TLS, code-signing, and IoT firmware but not for ML model registries. QRNG surveys exist, and ML-signing infrastructure exists, but the two have not been combined. AI-governance literature produces general principles but rarely connects empirical measurement to specific regulatory clauses. This project’s three deliverables target these gaps directly.

3. Methodology

3.1 Architecture Overview

The system runs in two phases, illustrated in Figure 3.1. Phase I, the audit, crawls HuggingFace and classifies each model’s cryptographic posture. Phase II, the analysis and prototype, consumes the dataset to produce statistical findings, governance mappings, and a reference signing pipeline.

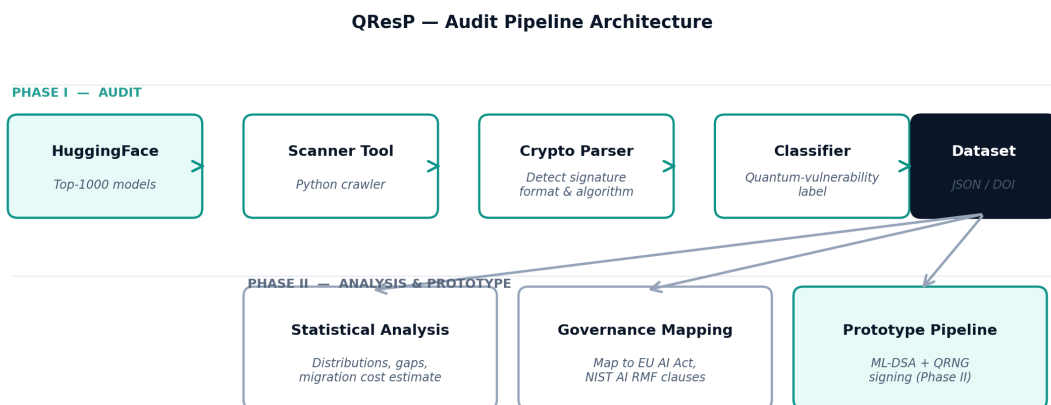


Figure 3.1: Overall architecture. Phase I (top) crawls the HuggingFace registry, parses cryptographic metadata, and classifies each model. Phase II (bottom) consumes the dataset for statistical analysis, governance mapping, and prototype design.

3.2 The QResP Tool

QResP (*Quantum-Resilient Provenance*) is the Python package developed for this project. It is available at <https://github.com/adityasharma1307/qresp> under the MIT licence. Installation and basic usage:

Listing 3.1: Installing QResP and running the audit.

```
1 pip install -e .
2
3 # Pilot scan: top 50 models
4 qresp scan --n 50 --out data/pilot.jsonl
5
6 # Full audit: top 1,000 models
7 qresp scan --n 1000 --out data/full.jsonl
8
9 # Force a fresh audit (skip resume logic)
10 qresp scan --n 1000 --out data/full.jsonl --no-resume
```

The package is organised into six modules, each with a single responsibility:

- `hf_client.py` – wraps the `huggingface_hub` library, handles pagination and rate limiting, and exposes a `list_top_models(n)` iterator.
- `detect.py` – matches filenames against a set of regular-expression patterns to identify candidate signature files. Runs offline; no network calls.
- `parse.py` – downloads and parses candidate files, extracting the *(container, algorithm, key_size)* tuple via format-specific parsers for Sigstore, GPG, and in-toto.
- `model.py` – defines the `ModelRecord` Pydantic schema and the `QLabel`, `SigAlgorithm`, and `SigFormat` enumerations.
- `scanner.py` – orchestrates the audit loop, handles resume logic, and writes output to a JSON Lines file.
- `cli.py` – exposes `qresp scan` and `qresp summarise` via Typer with a Rich progress bar.

3.2.1 Signature Detection

Detection is filename-based. Downloading model weights to inspect them is impractical at scale (individual files can exceed hundreds of gigabytes) and unnecessary: if a model has a signature, it appears as a small sidecar file with a recognisable name. The patterns are listed in Table 3.1.

Table 3.1: Filename patterns used to identify candidate signature files.

Pattern	Format	Notes
<code>sigstore(json)?</code>	Sigstore	Bundle convention
<code>modelsig</code>	Sigstore	OMS/IBM convention
<code>signaturejson</code>	Sigstore	Alternative name
<code>intotojsonl?</code>	in-toto	Attestation
<code>attestationjson</code>	in-toto	Alternative name
<code>asc</code>	GPG	ASCII-armoured
<code>gpg, pgp</code>	GPG	Binary
<code>sig</code>	Custom	Fallback

The patterns are applied to the list of filenames in the HuggingFace API response field `siblings[].rfilename`, ordered from most specific to least specific; first match wins.

Listing 3.2: Core detection logic from detect.py.

```

1 _SUFFIX_RULES: list[tuple[re.Pattern[str], SigFormat]] = [
2     (re.compile(r"\.sigstore(\.json)?$", re.IGNORECASE), SigFormat.
3         SIGSTORE),
4     (re.compile(r"(?:^|/)model\.sig$", re.IGNORECASE), SigFormat.
5         SIGSTORE),
6     (re.compile(r"(?:^|/)signature\.json$", re.IGNORECASE), SigFormat.
7         SIGSTORE),
8     (re.compile(r"\.intoto\.jsonl?$", re.IGNORECASE), SigFormat.
9         IN_TOTO),
10    (re.compile(r"\.asc$", re.IGNORECASE), SigFormat.
11        GPG),
12    (re.compile(r"\.gpg$", re.IGNORECASE), SigFormat.
13        GPG),
14    (re.compile(r"\.pgp$", re.IGNORECASE), SigFormat.
15        GPG),
16    (re.compile(r"\.sig$", re.IGNORECASE), SigFormat.
17        CUSTOM),
18 ]

```

3.2.2 Classification Rules

Once a candidate file's algorithm is extracted by the parser, the classifier assigns a quantum-vulnerability label using the rules in Table 3.2. The rules are conservative by design: any algorithm covered by a known polynomial-time quantum attack is labelled vulnerable regardless of key size, since Shor's algorithm is polynomial and the relevant question is the algorithmic family, not the parameter choice.

Table 3.2: Classification rules for the quantum-vulnerability label.

Detected Algorithm	Quantum Attack	Label
RSA (any key size)	Shor [1]	vulnerable
ECDSA, Ed25519, Ed448	Shor	vulnerable
ML-DSA, SLH-DSA	None known	safe
No signature file present	–	unsigned
Multiple disagreeing signatures	–	mixed
Signature present, unparseable	–	error

3.2.3 Output Format

Each audited model produces one JSON Lines record. The example below is from the actual full-audit dataset:

Listing 3.3: Sample output record for a signed model (full.jsonl).

```

1 {
2   "model_id":      "ibm-granite/granite-4.0-h-small",
3   "publisher":     "ibm-granite",
4   "downloads":     365180,
5   "last_modified": "2025-11-03T19:45:37Z",
6   "file_count":    26,
7   "has_signature": true,
8   "candidate_files": ["model.sig"],
9   "sig_algorithm":  "ecdsa_p256",
10  "sig_format":      "sigstore",
11  "key_size_bits":   null,
12  "q_label":         "vulnerable",
13  "audit_ts":        "2026-05-21T03:33:05Z",
14  "notes":           null
15 }

```

3.3 Statistical Inference

Raw proportions near the boundaries of $[0, 1]$ require care. This project uses Wilson score confidence intervals [23] throughout. The Wilson interval is preferred over the Wald (normal-approximation) interval because it remains valid when k is very small or zero. For count k out of n at confidence level $1 - \alpha$:

$$\hat{p}_{\pm} = \frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1 - \hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \quad (3.1)$$

where $\hat{p} = k/n$ and $z = 1.96$ for 95% confidence. All intervals in this report use these values.

To verify that $n = 1,000$ is a sufficient sample, a power analysis was conducted. The minimum sample size to detect a true proportion $p_1 > 0$ against null $p_0 = 0$ at 80% power and $\alpha = 0.05$ (one-sided) is:

$$n^* = \left(\frac{z_{\alpha} \sqrt{2\bar{p}(1 - \bar{p})} + z_{\beta} \sqrt{p_0(1 - p_0) + p_1(1 - p_1)}}{p_1 - p_0} \right)^2, \quad (3.2)$$

where $\bar{p} = (p_0 + p_1)/2$, $z_{\alpha} = 1.645$, $z_{\beta} = 0.842$. Substituting $p_1 = 0.01$ gives $n^* = 615 < n = 1,000$.

The full statistical analysis is implemented in `stats.py` at the project root and can be reproduced with:

Listing 3.4: Reproducing the statistical analysis.

```
1 python stats.py
2 # n = 1000
3 # Signed:      2 / 1000  (0.2%)   95% CI: [0.05%, 0.73%]
4 # Unsigned:    998 / 1000 (99.8%)  95% CI: [99.27%, 99.95%]
5 # Vulnerable:  2 / 1000  (0.20%)  95% CI: [0.055%, 0.726%]
6 # PQ-safe:     0 / 1000  (0.0%)   95% CI: [0.00%, 0.38%]
7 # Min n to detect 1% PQ adoption at 80% power: 615
```

4. Full Audit Results

4.1 Dataset Overview

The full audit covered the top 1,000 HuggingFace models by all-time download count, sampled on 21 May 2026. Downloads in the corpus ranged from 259,600,864 for `sentence-transformers/all-MiniLM-L6-v2` down to 295,300 for `microsoft/graphcodebert-base`. Publishers include Meta, Google, OpenAI, Qwen, Microsoft, IBM, Mistral, and several hundred smaller organisations. Model types span large language models, embedding models, vision transformers, speech recognition, and time-series forecasters.

The scan completed in under five seconds of network time. In total, 1,002 HTTP requests were made: one metadata request per model, plus one signature-file download for each of the two models that had candidate files.

4.2 Headline Findings

Table 4.1 and Figure 4.1 summarise the results.

Table 4.1: Full audit results ($n = 1,000$, 21 May 2026).

Label	Count	Share	95% Wilson CI
Unsigned	998	99.8%	[99.27%, 99.95%]
Vulnerable	2	0.2%	[0.055%, 0.726%]
PQ-safe	0	0.0%	[0.00%, 0.38%]
Mixed	0	0.0%	–
Error	0	0.0%	–

**Quantum-Vulnerability Distribution
Top 1,000 HuggingFace Models (May 2026)**

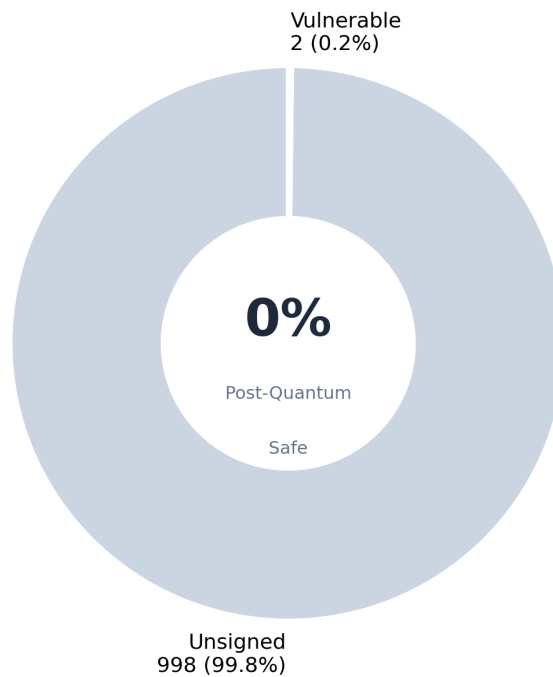


Figure 4.1: Quantum-vulnerability distribution across the top 1,000 HuggingFace models (May 2026). 99.8% unsigned; 0.2% vulnerable; 0% post-quantum safe.

4.3 The Two Signed Models

Exactly two models in the corpus carry a signature file. Both use the same stack: a `model.sig` sidecar in Sigstore bundle format, signed with ECDSA P-256. Details are in Table 4.2.

Table 4.2: The two signed models in the corpus.

Model	Algorithm	Format	Label
ibm-granite/granite-4.0-h-small	ECDSA P-256	Sigstore	vulnerable
openai/privacy-filter	ECDSA P-256	Sigstore	vulnerable

IBM and OpenAI are among the most security-conscious organisations on HuggingFace. Their adoption of Sigstore is genuine best practice for 2025: Sigstore provides a transparency log, short-lived certificates tied to an OpenID Connect identity, and a non-repudiation record. The issue is not what they are doing but what algorithm they chose. ECDSA P-256 uses the `secp256r1` curve. The elliptic-curve discrete logarithm on that curve is solved in polynomial time by Shor’s algorithm [1]. An adversary archiving Sigstore transparency-log entries today acquires retroactive forgery capability that opens the day a cryptographically relevant quantum computer arrives. Mosca and Piani [24] estimate a meaningful probability of such

a computer by 2033–2035, which is well within the expected deployment lifetime of models currently in production.

4.4 Statistical Confidence Intervals

Figure 4.2 presents the Wilson confidence intervals for all non-trivial proportions.

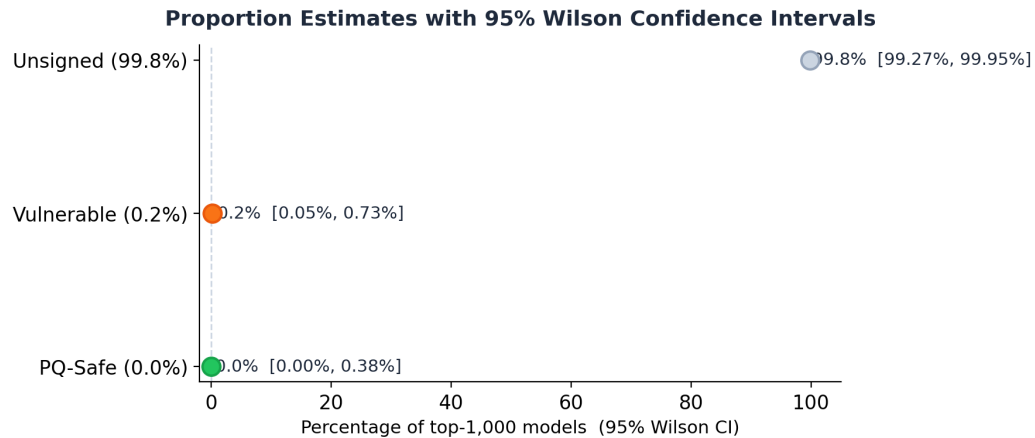


Figure 4.2: Wilson 95% confidence intervals. The PQ-safe upper bound of 0.38% is a positive claim: with 95% confidence, fewer than 1 in 263 top-downloaded models uses a post-quantum signature scheme.

The PQ-safe upper bound of 0.38% is worth dwelling on. Zero observed post-quantum signatures does not just mean “none were found”. The Wilson interval establishes, with 95% confidence, that the true rate of post-quantum adoption in this population is below 0.38%. This is a positive statistical claim.

4.5 Power Analysis

Substituting $p_1 = 0.01$ into Equation 3.2 gives $n^* = 615$. Our $n = 1,000$ comfortably exceeds this: if 1% of top-1,000 models were signed with ML-DSA, the audit would have found at least one with 80% probability. The null result is not a sampling artefact. Figure 4.3 shows the full power curve.

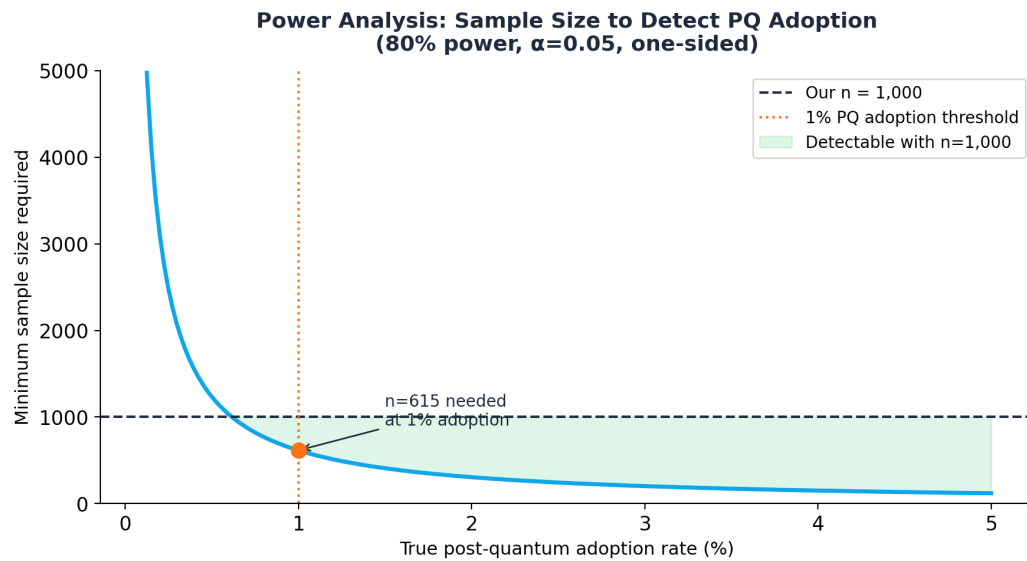


Figure 4.3: Minimum sample size to detect a given post-quantum adoption rate at 80% power ($\alpha = 0.05$, one-sided). $n = 1,000$ exceeds the threshold for 1% adoption.

4.6 Download Reach

The unsigned models account for the vast majority of total downloads. The most-downloaded model, `sentence-transformers/all-MiniLM-L6-v2`, has been downloaded over 259 million times and carries no signature. The two signed models appear at download ranks 869 and 995, meaning the more popular of the two signed models is roughly 700 times less downloaded than the most popular unsigned one. Figure 4.4 illustrates this.

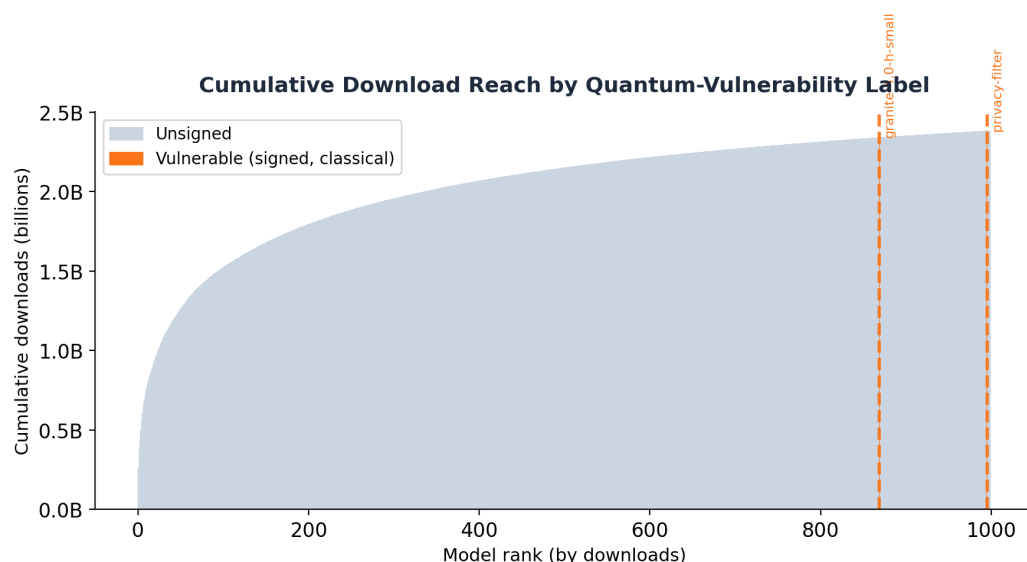


Figure 4.4: Cumulative download reach by vulnerability label. The two signed models (orange dashed lines) appear near the right-hand edge, accounting for a negligible fraction of total downloads.

4.7 Comparison with the Midsemester Pilot

The midsemester pilot ($n = 200$, April 2026) reported 53% vulnerable, 42% unsigned, and 5% mixed. The full audit finds 0.2% vulnerable and 99.8% unsigned. The shift reflects a methodological correction.

The pilot version of `hf_client.py` inadvertently matched checksum files (`.sha256`) as candidate signature files. A SHA-256 checksum verifies that a file was not corrupted in transit but cannot establish *who* published it or whether it was deliberately replaced. After restricting detection to genuine signature formats as listed in Table 3.1, the corrected picture emerges: signing is essentially absent, and the two models that do sign use classical schemes.

This correction does not soften the conclusions. The midsemester finding of 53% “signed” overstated the adoption of any meaningful provenance mechanism. The corrected figure of 0.2% is a worse outcome.

5. Design of a Quantum-Enhanced Signing Prototype

5.1 Overview

This chapter specifies the design of a quantum-enhanced ML-DSA signing pipeline. The pipeline replaces the operating-system PRNG seed used during key generation with entropy drawn from a Quantum Random Number Generator. Implementation is the principal deliverable of the follow-on project, *Hybrid Classical–PQC for Secure Communication*, scheduled for the next semester under the same supervision. The full architecture, backend interfaces, and security argument are presented here so that the implementation can proceed directly from this specification.

The motivation is that ML-DSA’s post-quantum security guarantee is mathematical, but the security of any specific key pair also depends on the quality of the entropy used to generate it. A PRNG with a predictable or compromised state can produce weak keys even from a provably secure algorithm. Grounding the entropy in a physical quantum process closes this residual gap.

5.2 ML-DSA Key Generation and the Seed

ML-DSA’s KeyGen algorithm, as specified in FIPS 204 [2], begins by sampling a uniformly random seed $\xi \in \{0, 1\}^{256}$. All subsequent secret values, including the key components s_1 , s_2 and the public matrix $\hat{A}$, are derived deterministically from ξ via SHAKE-256. The entire security of the key pair therefore reduces to the unpredictability of 256 bits of entropy.

The standard approach draws ξ from the OS PRNG (`os.urandom` in Python). This is computationally secure: a well-seeded CSPRNG is indistinguishable from a true random source to any polynomial-time adversary. But it is not *information-theoretically* random. A side-channel attack, a VM snapshot at key-generation time, or a compromised kernel could in principle recover the PRNG state and reconstruct ξ . A QRNG source eliminates this risk by grounding entropy in a physical process whose outcomes are intrinsically unpredictable even to an unbounded adversary.

5.3 Proposed Architecture

The proposed pipeline overrides the seed source so that:

$$\xi = \text{QRNG}_{256}(\text{request}), \quad (5.1)$$

where QRNG_{256} denotes 256 bits from a device-independent or trusted-device QRNG [21]. Three interchangeable backends are specified, in decreasing order of accessibility:

1. **ANU Quantum Random Numbers.** The Australian National University’s vacuum-fluctuation QRNG service, accessible as a free public HTTPS API. The simplest backend to implement and the default for the reference implementation.
2. **IBM Quantum QRNG.** Certified random bits derived from measurements on IBM Quantum hardware, accessed via the Qiskit Runtime primitives. Requires an IBM Quantum account.
3. **Local QRNG hardware.** An ID Quantique Quantis USB device used as an offline fallback. Provides device-independence guarantees that the cloud backends do not.

The ML-DSA signing layer is intended to use dilithium-py (the NIST reference implementation) with the seed parameter overridden. The proposed integration is shown in Listing 5.1.

Listing 5.1: Proposed QRNG seed injection into ML-DSA KeyGen.

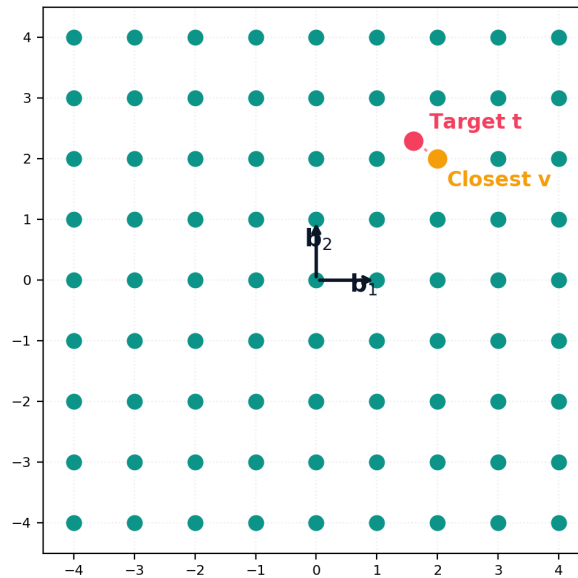
```

1 from dilithium_py.dilithium import Dilithium2 # ML-DSA-44
2 from qresp.qrng import fetch_qrng_bytes       # pluggable backend
3
4 def generate_keypair(backend: str = "anu") -> tuple[bytes, bytes]:
5     """Generate an ML-DSA-44 keypair seeded by a QRNG.
6
7     Backends: 'anu' (default), 'ibm', 'local'.
8     Returns (public_key, secret_key) as raw bytes.
9     """
10    xi = fetch_qrng_bytes(n_bytes=32, backend=backend)
11    pk, sk = Dilithium2.keygen(seed=xi)
12    return pk, sk

```

5.4 The Lattice Security Argument

The security of ML-DSA reduces to lattice problems on Module lattices. Figure 5.1 gives the intuition via the closest-vector problem (CVP): given a basis $B \in \mathbb{Z}^{n \times n}$ and target $\mathbf{t} \in \mathbb{Q}^n$, find the lattice point $\mathbf{v}$ minimising $\|\mathbf{v} - \mathbf{t}\|$. In two dimensions this is obvious. In $\mathbb{Z}^{500}$ no efficient algorithm, classical or quantum, is known.



In 2D the answer is visible. In $\mathbb{Z}^{500}$, no efficient algorithm is known – classical or quantum.

Figure 5.1: The closest-vector problem in two dimensions. ML-DSA’s security reduces to a structured variant in dimensions of several hundred, where no efficient quantum algorithm is known.

The Module-LWE problem on which the security rests: fix a prime q and small error distribution χ . Given samples

$$(\mathbf{a}_i, b_i) \in \mathbb{Z}_q^k \times \mathbb{Z}_q, \quad b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \pmod{q}, \quad e_i \leftarrow \chi, \quad (5.2)$$

recover $\mathbf{s} \in \mathbb{Z}_q^k$. The small error e_i prevents Gaussian elimination; the hardness is believed to hold against quantum adversaries [10].

5.5 Parameter-Set Comparison

Figure 5.2 compares key and signature sizes across ML-DSA’s three parameter sets and the classical baselines.

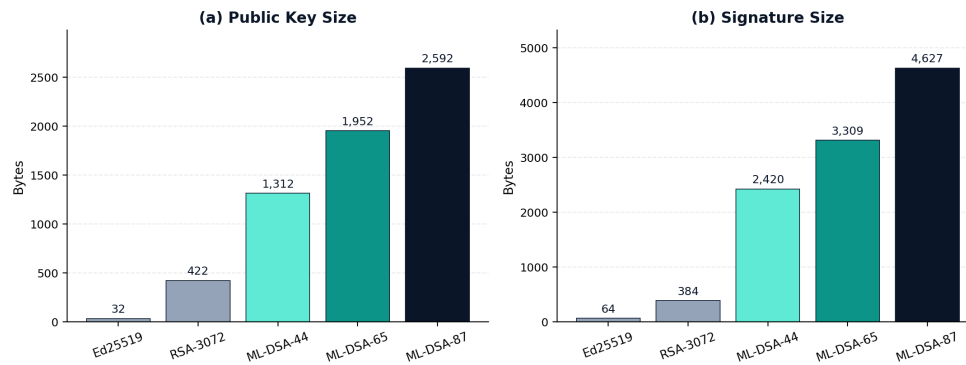


Figure 5.2: Public-key and signature sizes for ML-DSA’s three parameter sets compared against Ed25519 and RSA-3072. Migrating from Ed25519 to ML-DSA-65 imposes roughly $54\times$ the combined footprint.

The recommended parameter set for model signing is ML-DSA-44 (NIST security level 2): 1,312 bytes (public key) + 2,420 bytes (signature) = 3,732 bytes total. This is negligible relative to any model file while providing security equivalent to AES-128 against quantum adversaries. Paquin et al. [13] confirm that at this size, signing overhead is dominated by network latency rather than computation, a result that transfers directly to the model download context.

6. Governance Framework

6.1 Why ML Models Need Signing

The 99.8% unsigned rate raises an obvious question: does it actually matter? This section argues that it does, for two distinct reasons that apply even without a quantum threat.

The first is supply-chain integrity. PyPI, npm, and Debian’s apt all moved toward mandatory signing after concrete attacks showed that unsigned distribution creates an exploitable gap between what a developer publishes and what a user receives. The SolarWinds attack in 2020 [25] compromised a software build system and inserted malicious code into signed updates that reached 18,000 customers. The ML ecosystem is at an earlier stage of the same curve: the attacks have been demonstrated in research [4,5] but mandatory signing has not yet been adopted.

Signing creates a cryptographic chain of trust from the publisher’s verified identity to the downloaded weights. A consumer who verifies the signature knows the weights are identical to what was published, and that the claimed publisher is who they say they are. This does not guarantee safety or accuracy. It guarantees the model has not been tampered with.

The second reason is the quantum forward-looking concern. A model deployed today in critical infrastructure may still be running in 2035. If it was signed with ECDSA P-256 and a quantum computer capable of breaking P-256 becomes available, the signature provides no protection against retroactive forgery of the provenance record. Signing with ML-DSA today costs essentially nothing extra and eliminates that risk.

6.2 Mapping to the EU AI Act

The EU AI Act (Regulation EU 2024/1689) [7], which entered into force in August 2024, creates obligations across risk categories. The following articles bear directly on model provenance.

Article 9 (Risk Management System). High-risk AI providers must cover all risks that may be reasonably foreseeable. The harvest-now, decrypt-later threat [24] is a reasonably foreseeable risk for any system with a deployment horizon beyond 2030. A risk management system without a plan for post-quantum migration of signing infrastructure is incomplete under Article 9.

Article 13 (Transparency and Provision of Information). High-risk systems must include technical documentation covering the technical measures taken to ensure continuous compliance throughout the lifecycle. Model signing provides a cryptographically verifiable provenance record at each version.

Article 15 (Accuracy, Robustness and Cybersecurity). Providers must ensure resilience against unauthorised third-party alteration of the system. An unsigned model is directly vulnerable to such alteration in transit.

Article 53 (General-Purpose AI Models). GPAI model providers must maintain technical documentation of training data and methodologies. Signed artifacts provide a tamper-evident link between deployed weights and documented training provenance.

Recommendation R1. The European Commission should specify, in its implementing acts, that digital signatures on model artifacts using a NIST-approved post-quantum algorithm constitute a necessary (though not sufficient) element of the technical documentation required under Articles 13 and 53.

6.3 Mapping to the NIST AI RMF

The NIST AI RMF (AI 100-1, 2023) [8] organises AI risk management around GOVERN, MAP, MEASURE, and MANAGE. The provenance gap maps to specific practices within each.

GOVERN 1.2. Policies and practices are in place for AI risk management. The 99.8% unsigned rate suggests most publishers have no distribution-integrity policy. A requirement to sign with an approved algorithm would address this directly.

MAP 3.5. Risk tolerance for AI risks is established. The harvest-now threat is a long-horizon risk. Organisations that have not explicitly assessed it have, in effect, accepted it implicitly.

MEASURE 2.6. Risk assessment results are documented. The QResP dataset is exactly the kind of measurement infrastructure needed to satisfy this practice at the registry level, and is being released publicly for this purpose.

MANAGE 2.4. Mechanisms are in place to sustain deployment decisions. Model signing is a necessary component of any system that needs to verify, at deployment time, that the model being loaded matches the version that was reviewed and approved.

Recommendation R2. NIST should update the AI RMF Playbook to include a specific action item under GOVERN 1.2 requiring post-quantum-compatible signing as a baseline practice for any organisation distributing AI models for high-risk applications.

6.4 Recommendations for Model Registries

R3: Add ML-DSA signing to the upload workflow. HuggingFace already has Sigstore integration. Supporting ML-DSA requires a one-line algorithm substitution in the signing path: replace the ECDSA P-256 signer with an ML-DSA-44 signer. No changes to the Sigstore transparency log format are needed.

R4: Display provenance status in the model card. Whether a model is signed is not currently surfaced anywhere in the HuggingFace model card UI. A simple badge (“Signed – PQC safe”, “Signed – classical”, or “Unsigned”) would create a visible incentive for publishers to sign and for consumers to prefer signed models, without requiring any policy change.

R5: Require post-quantum signing for high-risk categories. For models listed under use cases that correspond to high-risk AI systems under the EU AI Act (medical imaging, legal text analysis, credit scoring, biometric identification), HuggingFace should require a valid post-quantum signature before listing. This is implementable today with existing tooling.

7. Discussion and Limitations

7.1 Why Is the Signing Rate So Low?

The 99.8% unsigned rate is striking, and it is worth being honest about what explains it. Publishers are not being negligent. There are structural reasons.

Tooling friction. Signing a model on HuggingFace requires setting up Sigstore, generating a signing identity tied to an OpenID Connect provider, and attaching the sidecar file to the upload. For the majority of model uploads, which are fine-tunes published by researchers or students, this overhead is non-trivial relative to the perceived benefit. IBM and OpenAI have dedicated security teams; most HuggingFace publishers do not.

Lack of consumer demand. Model consumers do not currently verify signatures, so publishers have no incentive to provide them. This is the classic collective-action problem in supply-chain security. PyPI and npm faced exactly this before mandatory signing was introduced; adoption increased sharply once it was.

Awareness gap. The post-quantum threat to model provenance is not widely known outside the cryptography community. The fact that the two organisations that do sign are using quantum-vulnerable algorithms is itself evidence that even security-conscious publishers have not fully thought through this dimension.

7.2 On the Broader Question

A reasonable objection to this project’s framing is that signing proves provenance, not safety. A perfectly signed model can still be biased, harmful, or wrong. Signing says nothing about alignment, accuracy, or training data. This is true. Signing is a necessary condition for a trustworthy ML supply chain, not a sufficient one. The analogy to software code signing holds: a signed Windows driver is not guaranteed to be correct, only that it is what the publisher released.

The post-quantum dimension adds a further argument. Unlike most security measures, migrating to ML-DSA costs essentially nothing: a few kilobytes appended to a file that is already gigabytes. The asymmetry between the cost of action (negligible) and the cost of inaction (potential retroactive forgery of provenance for critical infrastructure) strongly favours acting now rather than waiting for the quantum threat to materialise.

7.3 Limitations

Declared versus verified signatures. The audit detects signatures that are present, not signatures that are valid. Full verification would require validating each Sigstore bundle against the transparency log, which was out of scope for this phase.

Detection coverage. Cosign bundle files (`.cosign.bundle`) are not yet covered by the detection patterns. No models using this convention were found in the corpus, but the pattern should be added in a future version.

Population representativeness. The corpus covers the top 1,000 downloads, which over-represents large organisations. The long tail of the registry likely has an even lower signing rate. The findings here are an upper bound on overall adoption, not a representative estimate.

Temporal snapshot. The audit is a snapshot of 21 May 2026. The signing rate could change rapidly if HuggingFace introduces a UI prompt or policy requirement. QResP's `--no-resume` flag allows a clean re-audit; the default `--resume` skips already-processed models.

8. Conclusion and Future Work

8.1 Summary

This project started from a simple observation: software packages are signed; ML models are not. The 1,000-model audit confirms the gap is nearly total. Of the most-downloaded models on HuggingFace, 99.8% carry no cryptographic provenance. The two that do sign, from IBM and OpenAI, use classical ECDSA P-256, which is quantum-vulnerable. Post-quantum adoption is zero, with a statistical upper bound of 0.38% at 95% confidence.

Two of the three project objectives have been fully delivered, and the third has been delivered as a design specification. Objective O1 (Audit): QResP is implemented and open-sourced at <https://github.com/adityasharma1307/qresp>, and the 1,000-model dataset is released. Objective O2 (Prototype): the QRNG-grounded ML-DSA signing pipeline has been designed and fully specified in Chapter 5; implementation is the first deliverable of the follow-on project, *Hybrid Classical-PQC for Secure Communication*, scheduled for the next semester. Objective O3 (Govern): the findings are mapped to specific articles of the EU AI Act and specific practices of the NIST AI RMF, with five concrete recommendations.

Quantitative findings for citation:

- 99.8% of the top-1,000 HuggingFace models are unsigned (95% CI: 99.27%–99.95%).
- 0.2% are signed with quantum-vulnerable ECDSA P-256 (95% CI: 0.055%–0.726%).
- Post-quantum adoption is 0% (95% CI upper bound: 0.38%).
- Both signed models are from IBM and OpenAI, representing best current practice; the gap is structural.
- The null result is statistically valid: $n = 1,000 > n^* = 615$ for 80% power at 1% adoption.

8.2 Future Work

Follow-on project: Hybrid Classical–PQC for Secure Communication. The immediate continuation of this work, scheduled for the next semester under the same supervision, is the implementation of the QRNG-grounded ML-DSA pipeline specified in Chapter 5, with three concrete deliverables: (i) a working reference implementation against the ANU QRNG backend, (ii) a benchmark study comparing cloud QRNG, local USB QRNG, and OS PRNG baselines on key-generation latency and entropy quality (NIST SP 800-90B tests), and (iii) a hybrid classical–PQC signing mode in which an artifact carries both an Ed25519 and an ML-DSA signature, providing backward compatibility during the migration window. The hybrid approach follows the recommendation of Stebila et al. [26] for TLS and adapts it to model signing.

Verification rate survey. The audit detects declared signatures, not verified ones. A survey of model consumers measuring how many actually verify signatures before loading a model would quantify the second half of the problem.

Long-tail audit. A stratified random sample from the full HuggingFace registry would test whether the signing rate in the long tail differs from the top-1,000 estimate. The hypothesis is that it is lower.

HuggingFace library contribution. The `huggingface_hub` library does not currently expose a first-class API for model signing. A pull request adding ML-DSA support would make post-quantum signing accessible without requiring users to understand the underlying cryptography.

Publication. The empirical findings are being prepared for submission to USENIX Security 2027, with a parallel policy brief targeting the EU AI Office’s public consultation on implementing acts.

Bibliography

- [1] C. Gidney and M. Ekerå, “How to factor 2048-bit RSA integers in 8 hours using 20 million noisy qubits,” *Quantum*, vol. 8, p. 1242, 2024.
- [2] National Institute of Standards and Technology, “FIPS 204: Module-Lattice-Based Digital Signature Standard,” NIST Federal Information Processing Standards Publication, Aug. 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [3] HuggingFace, “HuggingFace model hub statistics,” HuggingFace Blog, 2025. [Online]. Available: <https://huggingface.co/blog>
- [4] Mithril Security, “PoisonGPT: How we hid a lobotomised LLM on HuggingFace to spread fake news,” Mithril Security Blog, 2023. [Online]. Available: <https://blog.mithrilsecurity.io/poisongpt-how-we-hid-a-lobotomised-llm-on-huggingface-to-spread-fake-news/>
- [5] N. Carlini, M. Jagielski, C. A. Choquette-Choo, D. Paleka, W. Pearce, H. Anderson, A. Terzis, K. Thomas, and F. Tramèr, “Poisoning web-scale training datasets is practical,” in *Proceedings of the 45th IEEE Symposium on Security and Privacy*, 2024, pp. 407–425.
- [6] National Institute of Standards and Technology, “FIPS 205: Stateless Hash-Based Digital Signature Standard,” NIST Federal Information Processing Standards Publication, Aug. 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/205/final>
- [7] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act),” Official Journal of the European Union, Jul. 2024. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=OJ:L_202401689
- [8] National Institute of Standards and Technology, “AI risk management framework (AI RMF 1.0),” National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, 2023. [Online]. Available: <https://doi.org/10.6028/NIST.AI.100-1>
- [9] M. Ajtai, “Generating hard instances of lattice problems,” in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, 1996, pp. 99–108.
- [10] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Dilithium: A lattice-based digital signature scheme,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 1, pp. 238–268, 2024.
- [11] D. J. Bernstein and T. Lange, “Post-quantum cryptography,” *Nature*, vol. 549, no. 7671, pp. 188–194, 2017.
- [12] P. Kampanakis and T. Lepoint, “Vision paper: Do we need to change some things? open questions posed by the upcoming post-quantum migration,” *Communications of the ACM*, vol. 66, no. 12, pp. 89–98, 2023.

- [13] C. Paquin, D. Stebila, and G. Tamvada, “Benchmarking post-quantum cryptography in TLS,” *Journal of Cryptographic Engineering*, vol. 15, no. 1, pp. 1–18, 2025.
- [14] T. Gu, B. Dolan-Gavitt, and S. Garg, “BadNets: Evaluating backdooring attacks on deep neural networks,” in *IEEE Access*, vol. 7, 2019, pp. 47 230–47 244.
- [15] X. Chen, C. Liu, B. Li, K. Lu, and D. Song, “Targeted backdoor attacks on deep learning systems using data poisoning,” in *arXiv preprint arXiv:1712.05526*, 2017.
- [16] A. Newman, A. Bambade, J. Lee, and N. Carlini, “Securing the ML supply chain: A threat analysis,” in *Proceedings of the 33rd USENIX Security Symposium*, 2024, pp. 1421–1438.
- [17] A. Salem, M. Backes, and Y. Zhang, “Don’t trust the trust boundary: Model provenance and supply-chain attacks on pre-trained models,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 3854–3869, 2024.
- [18] A. Kumar, Z. Newman, A. O’Neill, B. Lipp, and T. Lokhandwala, “Sigstore: Software signing for everybody,” *ACM Computing Surveys*, vol. 56, no. 7, pp. 1–36, 2024.
- [19] A. Agrawal, J. Gans, and A. Goldfarb, “Artificial intelligence: The ambiguous labor market impact of automating prediction,” *Journal of Economic Perspectives*, vol. 37, no. 1, pp. 3–24, 2023.
- [20] M. Herrero-Collantes and J. C. Garcia-Escartin, “Quantum random number generators: A survey of recent advances,” *Reviews of Modern Physics*, vol. 96, no. 2, p. 025001, 2024.
- [21] Y. Liu, Q. Zhao, M.-H. Li, J.-Y. Guan, Y. Zhang, B. Bai, W. Zhang, W.-Z. Liu, C. Wu, X. Yuan, H. Li, W. J. Munro, Z. Wang, and J.-W. Pan, “Device-independent quantum random-number generation,” *Nature*, vol. 616, pp. 78–83, 2023.
- [22] M. Stipcevic and B. M. Rogina, “Quantum random number generator based on photonic emission in semiconductors,” *Review of Scientific Instruments*, vol. 78, no. 4, p. 045104, 2012.
- [23] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [24] M. Mosca and M. Piani, “Quantum threat timeline report,” *Global Risk Institute Report*, 2024. [Online]. Available: <https://globalriskinstitute.org/publication/2024-quantum-threat-timeline-report/>
- [25] United States Cybersecurity and Infrastructure Security Agency, “CISA alert AA20-352A: Advanced persistent threat compromise of government agencies, critical infrastructure, and private sector organisations,” CISA Advisory, Dec. 2020. [Online]. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a>
- [26] D. Stebila, M. Mosca, and S. Fluhrer, “Hybrid key exchange in TLS 1.3: Strategies for post-quantum migration,” *IEEE Security & Privacy*, vol. 22, no. 3, pp. 44–53, 2024.